



Blockchain Principles and Applications

Amir Mahdi Sadeghzadeh, PhD

Data and Network Security Lab (DNSL)
Trustworthy and Secure AI Lab (TSAIL)

Rollups

Scaling the blockchain: the problem

Transaction rates (Tx/sec):

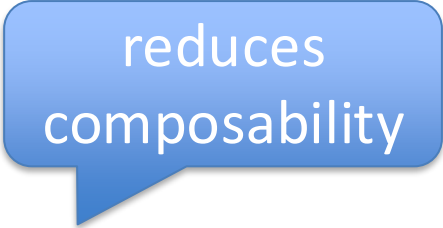
- Bitcoin: can process about **7 (Tx/sec)**
 - Ethereum: can process about **15 (Tx/sec)**
 - The visa network: can process up to **24,000 (Tx/sec)**
- Tx Fees fluctuate:
2\$ to 60\$ for simple Tx

Can we scale blockchains to visa speeds? ... with low Tx fees

How to process more Tx per second

Many ideas:

- Use a faster consensus protocol
- Parallelize: split the chain into independent **shards**
- Today: Rollups, move the work somewhere else
- Payment channels: reduce the need to touch the chain
 - Requires locking up funds; mostly designed for payments.



reduces
composability

Desired properties

- L2 solution
 - Lightweight protocol via smart contracts
 - No change in L1
- Security and data availability
 - Reuse Ethereum security
 - Avoid introducing additional trust
- EVM compatibility
 - Easy to migrate a Dapp from L1 to L2
 - Reduced Tx fees and increased Tx throughput compared to L1

Recall: a basic layer-1 blockchain

Can handle 15 Tx/sec ...



Tx_A



Tx_B



⋮

World state: balances, storage, etc.

A layer-1 blockchain
(e.g., Ethereum)

current world state



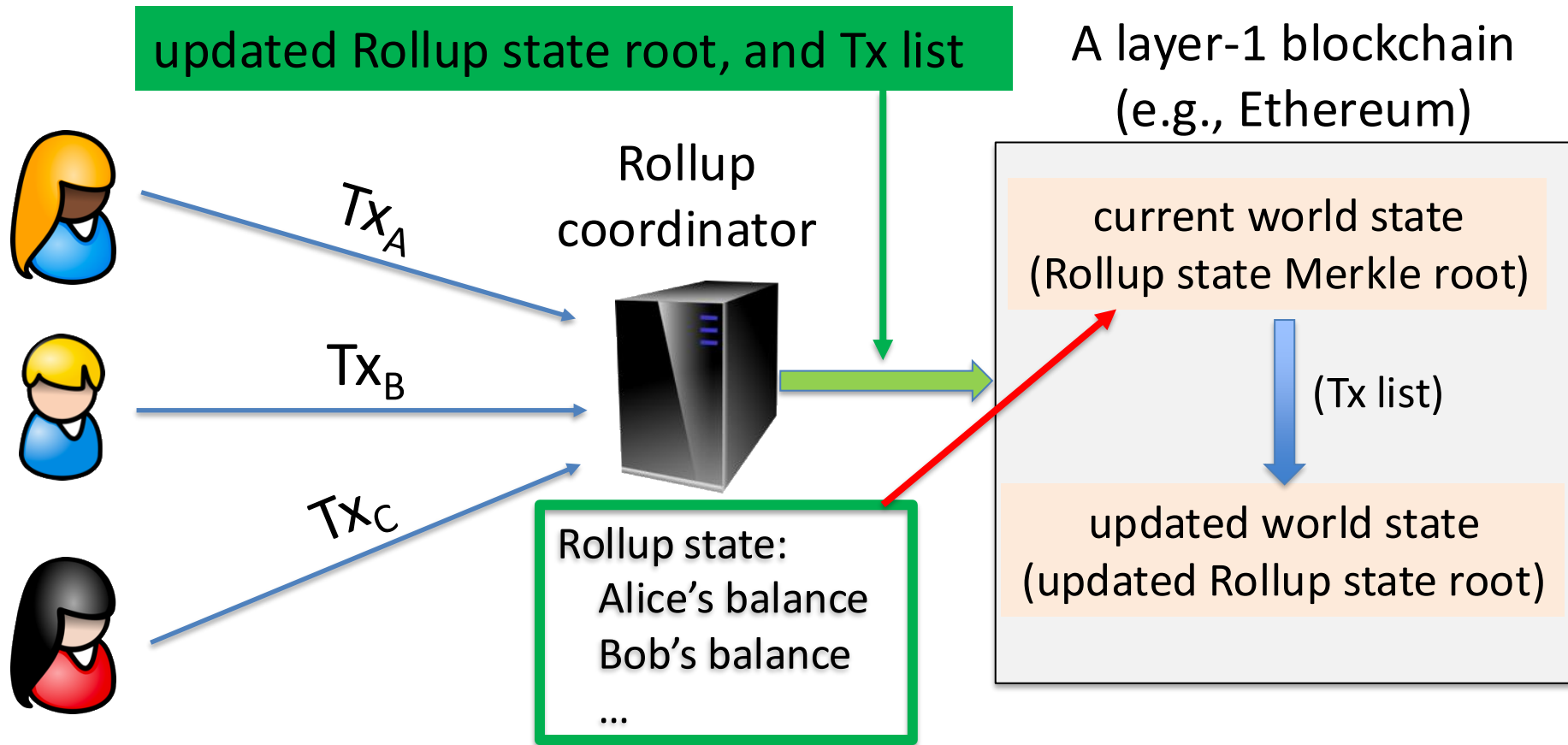
updated world state



updated world state



Rollup idea 1: batch many Tx into one



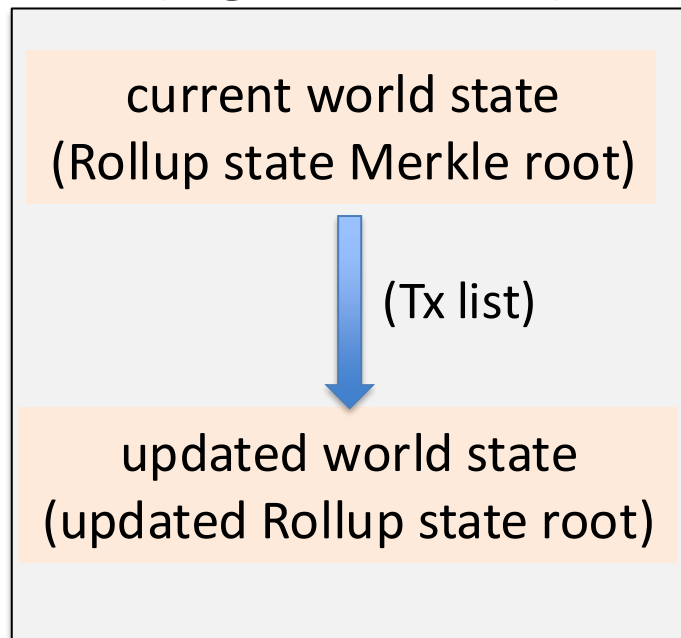
Rollup idea 1: batch many Tx into one

Key point:

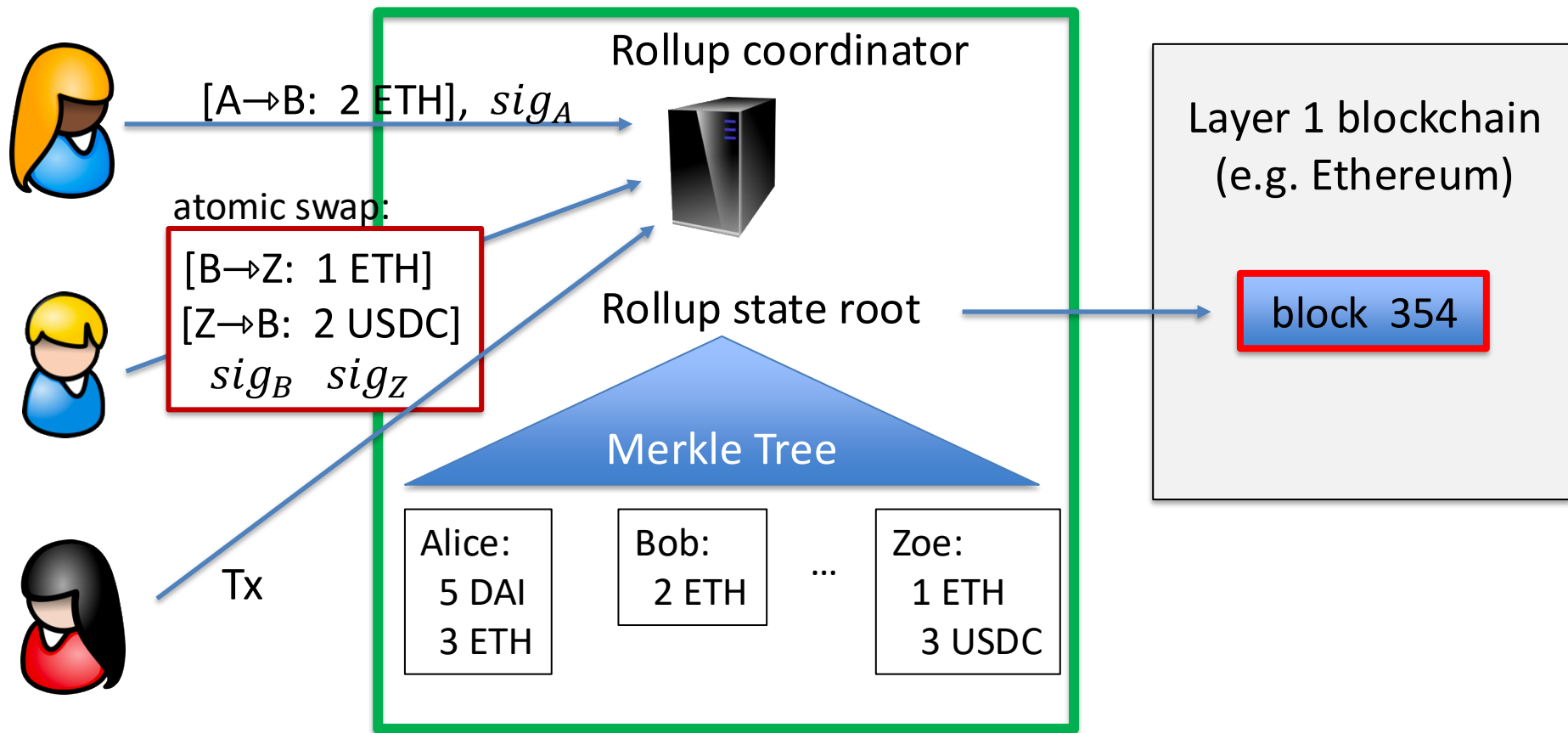
- *Hundreds* of transactions on Rollup state are batched into a *single* transaction on layer-1
⇒ 100x speed up in Tx/sec
- Let's see how ...

Rollup state:
Alice's balance
Bob's balance
...

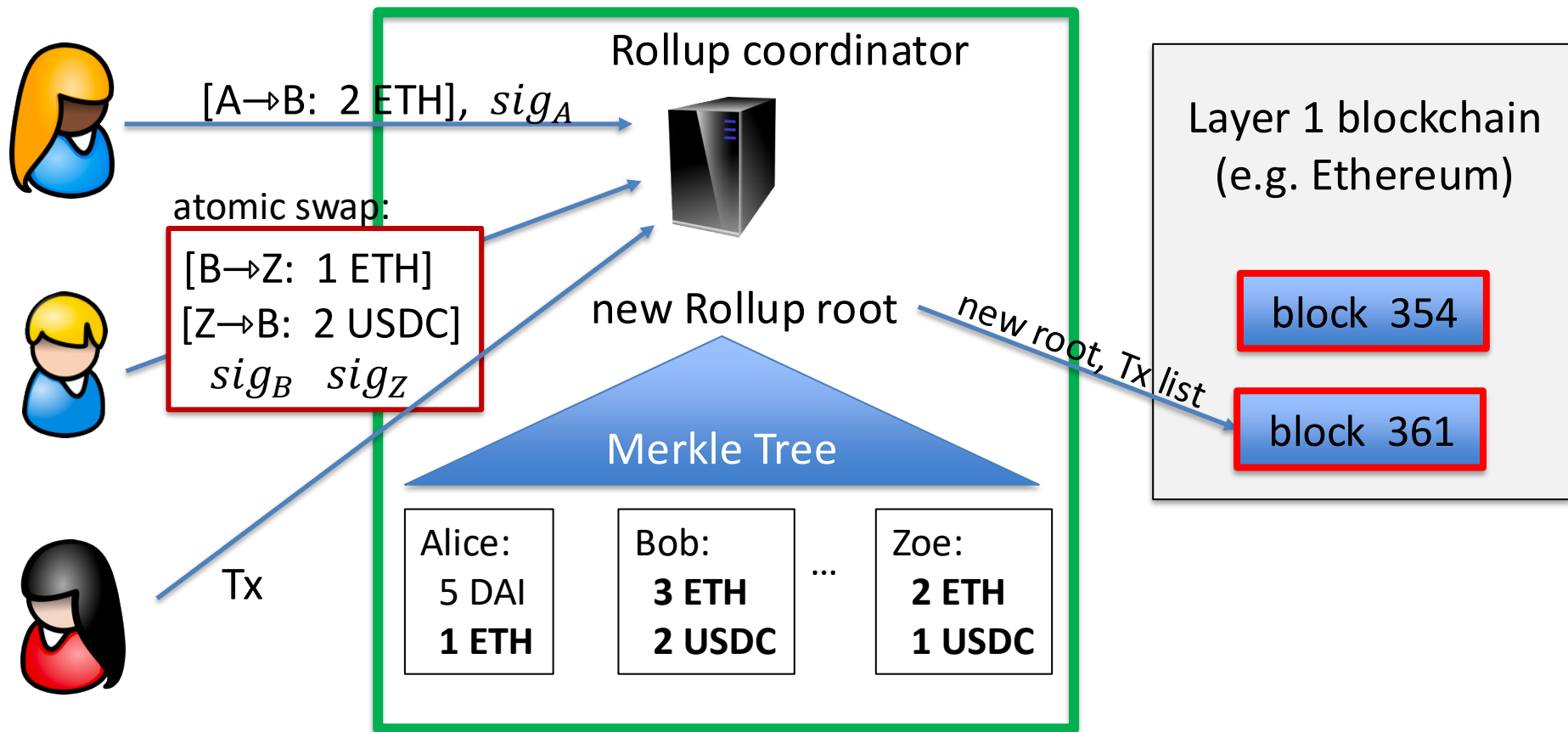
A layer-1 blockchain
(e.g., Ethereum)



Rollup operation (simplified)



Rollup operation (simplified)



In more detail

Rollup contract on
layer-1 holds assets
of all Rollup accounts
(and Merkle state root)

Rollup state (L2)

Alice: 4 ETH, 1 DAI	Bob: 3 ETH, 2 DAI	...
-------------------------------	-----------------------------	-----

(coordinator stores state)

Alice: state	Bob: state	Uniswap: state	Rollup contract: 7 ETH, 3 DAI, root	...
-----------------	---------------	-------------------	---	-----

Layer-1 blockchain (L1)

Transfers inside Rollup are easy (L2 → L2)



[A→B: 2 ETH], sig_A

(with hundreds of Tx)

Rollup state (L2)

Alice:

4 ETH, 1 DAI

Bob:

3 ETH, 2 DAI

...

Alice:
state

Bob:
state

Uniswap:
state

Rollup contract:
7 ETH, 3 DAI, root

...

Layer-1 blockchain (L1)

Transfers inside Rollup are easy (L2 → L2)

Coordinator updates root on Rollup contract



[A→B: 2 ETH], sig_A

(with hundreds of Tx)

Rollup state (L2)

Alice:

2 ETH, 1 DAI

Bob:

5 ETH, 2 DAI

...

new Merkle root,
Tx list

Alice:
state

Bob:
state

Uniswap:
state

Rollup contract:
7 ETH, 3 DAI, root

...

Layer-1 blockchain (L1)

Transferring funds into Rollup (L1 → L2)

Alice issues an L1 Tx: slow and expensive

Rollup state (L2)



[2 ETH], σ_A

Alice: 2 ETH, 1 DAI	Bob: 5 ETH, 2 DAI	...
-------------------------------	-----------------------------	-----

Alice: state	Bob: state	Uniswap: state	Rollup contract: 7 ETH, 3 DAI, root	...
-----------------	---------------	-------------------	---	-----

2 ETH

Layer-1 blockchain (L1)

Transferring funds into Rollup (L1 → L2)

Alice issues an L1 Tx: slow and expensive



[2 ETH], sig_A

Rollup state (L2)

Alice: 4 ETH, 1 DAI	Bob: 5 ETH, 2 DAI	...
-------------------------------	-----------------------------	-----

new Merkle root,
Tx list

Alice: state	Bob: state	Uniswap: state	Rollup contract: 9 ETH, 3 DAI, root	...
-----------------	---------------	-------------------	---	-----

2 ETH

Layer-1 blockchain (L1)

Transferring funds out of Rollup (L2 → L1)

Requires extra gas on L1 to process transfer



[withdraw 4 ETH], sig_A
(plus hundreds of Tx)

Rollup state

Alice: 4 ETH, 1 DAI	Bob: 5 ETH, 2 DAI	...
-------------------------------	----------------------	-----

Alice: state	Bob: state	Uniswap: state	Rollup contract: 9 ETH, 3 DAI, root	...
-----------------	---------------	-------------------	---	-----

Layer-1 blockchain (L1)

Transferring funds out of Rollup (L2 → L1)

Requires extra gas on L1 to process transfer



[withdraw 4 ETH], sig_A
(plus hundreds of Tx)

Rollup state

Alice: 0 ETH, 1 DAI	Bob: 5 ETH, 2 DAI	...
-------------------------------	----------------------	-----

new Merkle root,
Tx list

Alice: state	Bob: state	Uniswap: state	Rollup contract: 5 ETH, 3 DAI, root	...
-----------------	---------------	-------------------	---	-----

4 ETH

Layer-1 blockchain (L1)

Summary: transferring Rollup assets

Transactions within a Rollup are easy:

- Batch settlement on L1 network (e.g., Ethereum)

Moving funds into or out of Rollup system ($L1 \Leftrightarrow L2$) is expensive:

- Requires posting more data on L1 network $\Rightarrow$ higher Tx fees.

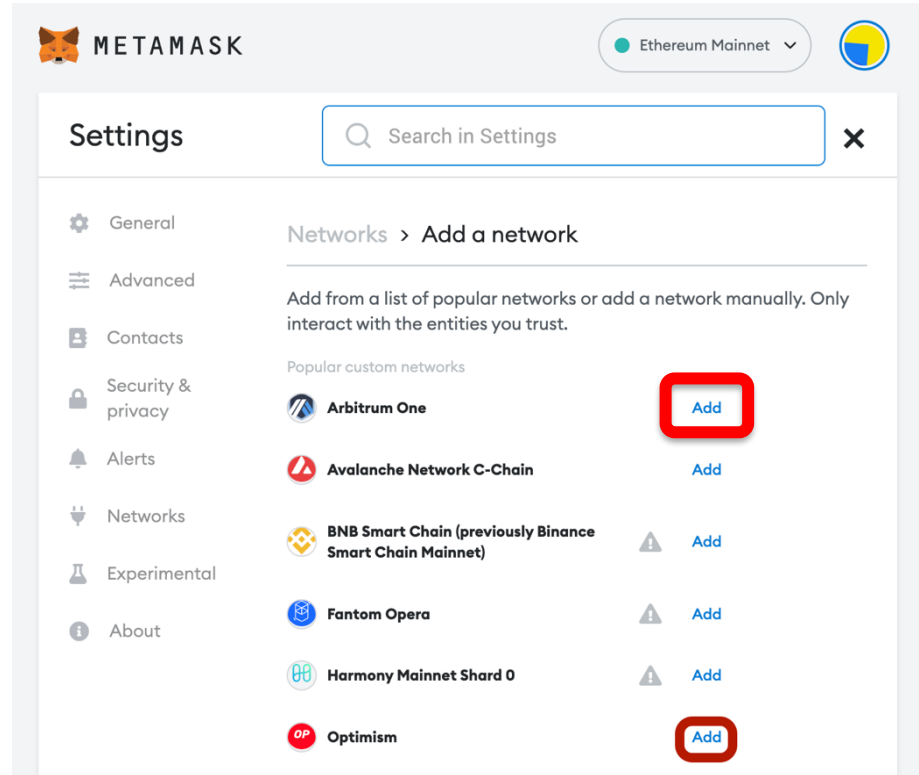
Moving funds from one Rollup system to another ($L2 \Leftrightarrow L2$)

- Either via L1 network (expensive),
or via a direct $L2 \Leftrightarrow L2$ bridge (cheap)

How to send Tx to the coordinator

Enduser configures its wallet to send Tx to the RPC points of the selected Rollup.

(by default Metamask sends Tx to the Ethereum Mainnet RPC points)



The role of the Coordinator

The Coordinator has multiple tasks:

- **Sequence** incoming Tx from Rollup users into a stream of Tx
 - ⇒ can extract MEV from searchers, in addition to Tx fees
 - ⇒ very profitable to be a Rollup coordinator
- **Execute** the stream of Tx on the latest Rollup state
- **Push updates** to the L1 chain

Shared coordinator: one coordinator for multiple Rollups

Coordinator architectures

A centralized coordinator:

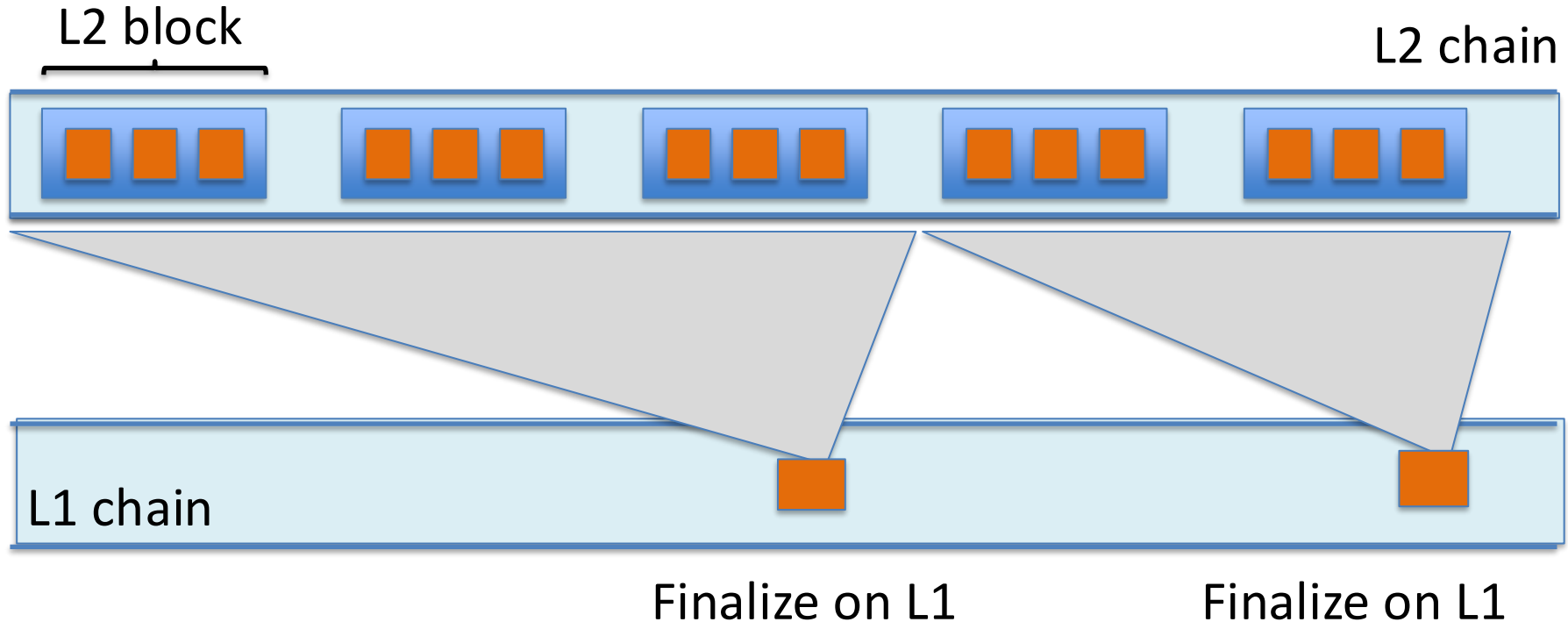
- availability and censorship concerns,
... but cannot steal assets

A decentralized coordinator:

- a set of parties that run a fast consensus protocol
- At every epoch one party is chosen to sequence, execute, and push updates to the L1

Importantly: L1 provides ground truth of the Rollup state

Tx rate on L2 is higher than on L1



An example (StarkNet -- using validity proofs)

Block				
Number ⓘ ⌵	Hash ⓘ	Status ⓘ ⌵	Num. of Txs ⓘ	Age ⓘ ⌵ ⌵
PENDING	PENDING	PENDING	64	3min
13011	0x0432...2380	ACCEPTED_ON_L2	82	8min
13010	0x0492...f0d1	ACCEPTED_ON_L2	122	15min
13009	0x0081...b7af	ACCEPTED_ON_L2	127	24min
		...		
12868	0x060c...15eb	ACCEPTED_ON_L2	58	8h
12867	0x0654...3b0f	ACCEPTED_ON_L1	72	9h
12866	0x0779...57d6	ACCEPTED_ON_L1	63	9h
12865	0x06ae...943f	ACCEPTED_ON_L1	97	9h

Tx posted on L1 (Ethereum) about every eight hours

Source: starkscan.co

Not so simple ...

Problems ...

Problem 1: what if coordinator is dishonest?

- It could steal funds from the Rollup contract
- It could issue fake Tx on behalf of users

Problem 2: what if coordinator stops providing service?

- If Rollup state is lost, how can users issue Tx?
 - ... can't compute updated Rollup Merkle root.

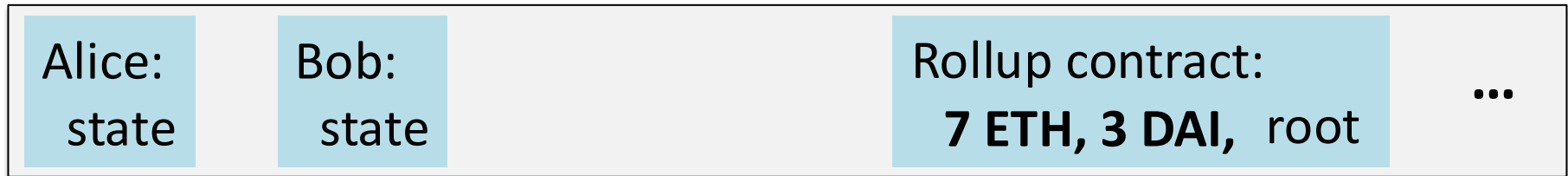
Problem 1: what if coordinator is dishonest?

Can coordinator steal funds from Rollup users?

No! L1 chain verifies that Rollup state updates are valid.

⇒ all Tx are valid and properly signed by the Rollup users

Challenge: how to do this cheaply ?? (with little gas on L1)



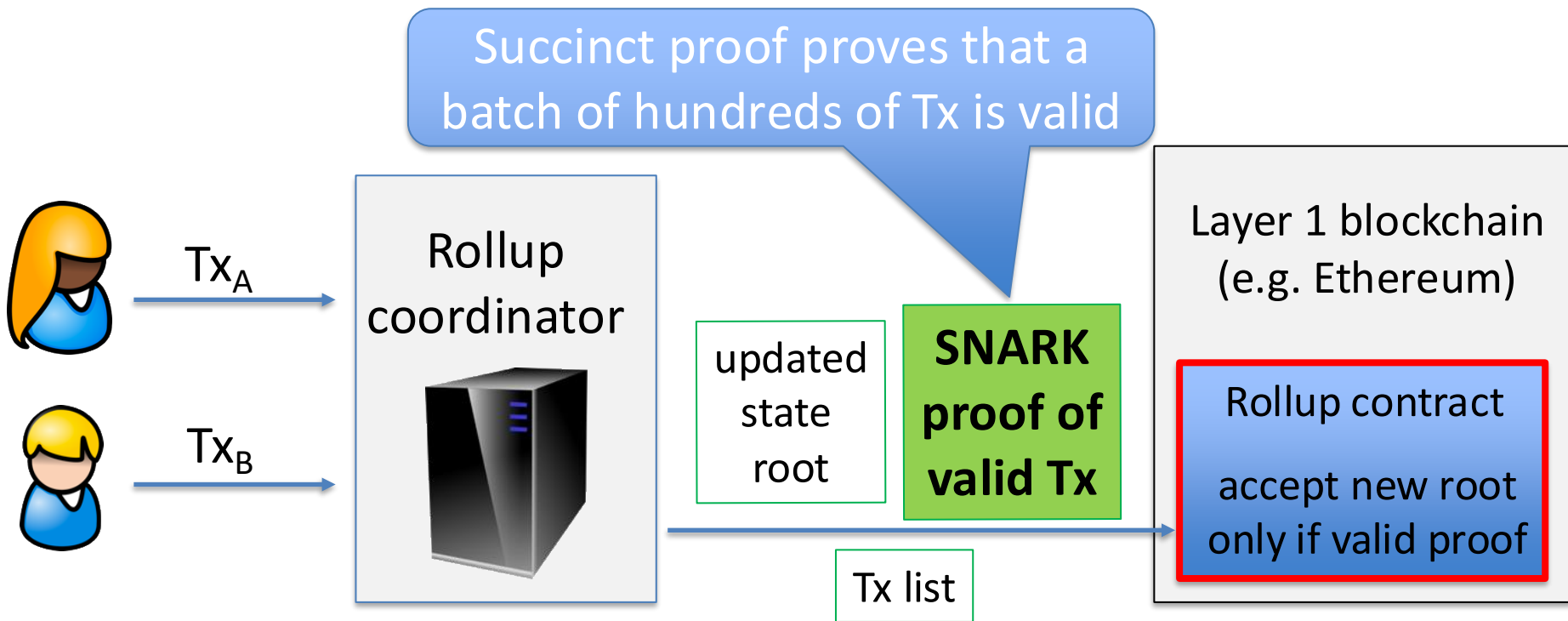
Layer-1 blockchain (L1)

Two different approaches to Rollups

- Optimistic rollup
 - Go ahead and commit the transactions assuming honest assertion
 - Depend on feedback to detect and address fraud
- ZK rollup
 - Compress state with an upfront proof of correctness
 - Verification is easier than the compression stage

Verifying Rollup state updates

Approach 1: validity proofs (called a zk-Rollup)



What the SNARK proof proves

SNARK proof is **short** and **fast** to verify:

⇒ Cheap to verify proof on the slow L1 chain (with EVM support)
(usually not a zero knowledge proof)

Public statement: (old state root, new state root, Tx list)

Witness: (state of each touched account pre- and post- batch,
Merkle proofs for touched accounts, user sigs)

SNARK proof proves that:

- (1) all user sigs on Tx are valid, (2) all Merkle proofs are valid,
- (3) post-state is the result of applying Tx list to pre-state

zkEVM

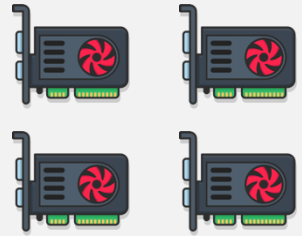
When a contract (e.g. Uniswap) runs on a Rollup:

- coordinator builds a SNARK proof of correct execution of an EVM program $\Rightarrow$ called a **zkEVM**
- Generating proof is a heavyweight computation
... verifying proof is fast

Flavors of a zkEVM:

- Prove that EVM bytecode ran correctly
(Polygon zkEVM, Scroll)
- Compile Solidity to a SNARK-friendly circuit
(MatterLabs)

Rollup
coordinator



(lots of GPUs)

The end result

Rollup contract on L1 ensures coordinator cannot cheat:

- all submitted Tx must have been properly signed by users
- all state updates are valid

⇒ Rollup contract on L1 will accept any update with a valid proof

⇒ Anyone can act as a coordinator (with enough compute power)

Verifying Rollup state updates

Approach 2: **fault proofs** (called an **optimistic Rollup**)

- Coordinator deposits stake in escrow on L1 Rollup contract
- Operation: Coordinator submits state updates to L1 w/o a proof
- If update is invalid: anyone has seven days to submit a fault proof
 - Successful fault proof means coordinator gets *slashed* on L1
 - Unsuccessful fault proof costs complainer a fee

Challenge: how to prove a fault to the Rollup contract on L1 ??

Naively: L1 can re-execute all Tx in batch $\Rightarrow$ expensive and slow

Fault Proof game



coordinator

claimed
post-root

pre-root

Tx list



fault claim

different
post-root

Coordinator computes Merkle
tree of all states.
Sends Merkle root to L1

pre-
state



break computation into small steps

post-
state

Fault Proof game



coordinator

claimed
 $state_n$

pre-root

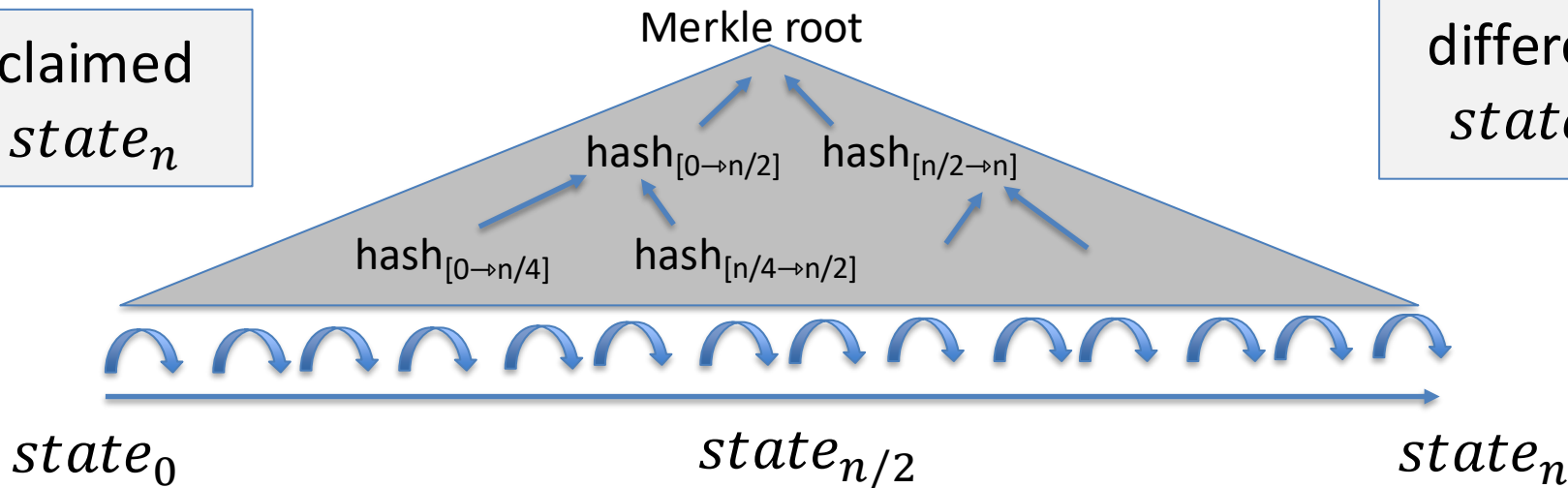
Tx list

we know $state_n \neq state'_n$



fault claim

different
 $state'_n$



Fault Proof game: binary search



coordinator

claimed
 $state_n$

pre-root

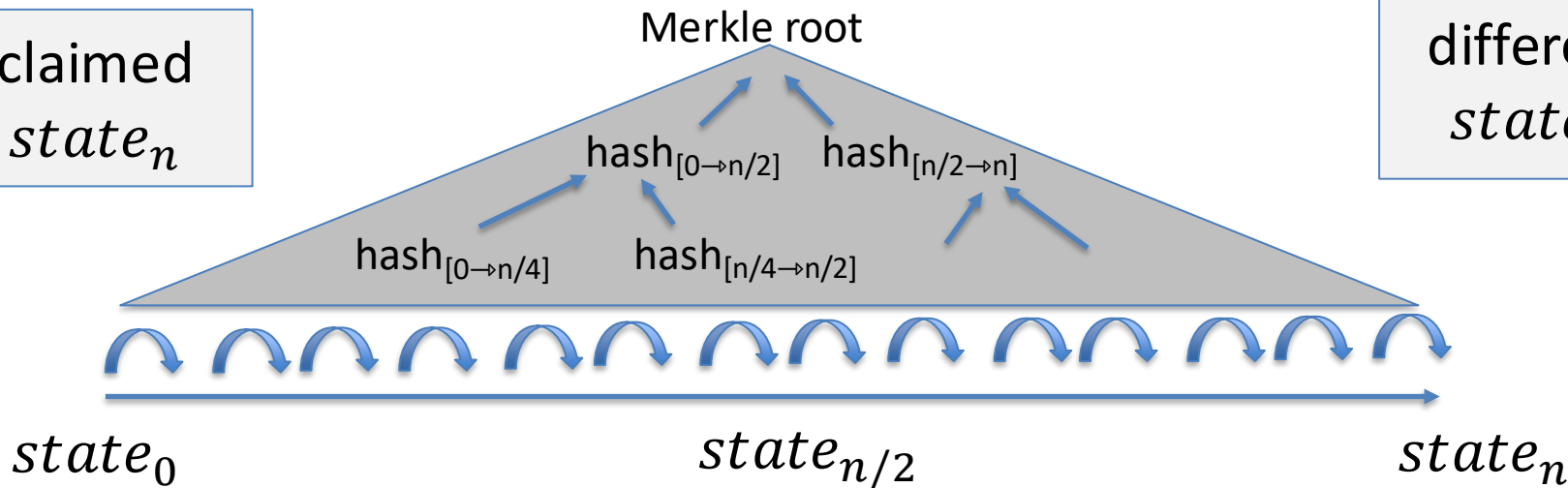
Tx list



fault claim

Suppose $state_{n/2} \neq state'_{n/2}$

different
 $state'_n$



Fault Proof game: binary search



coordinator

claimed
 $state_n$

pre-root

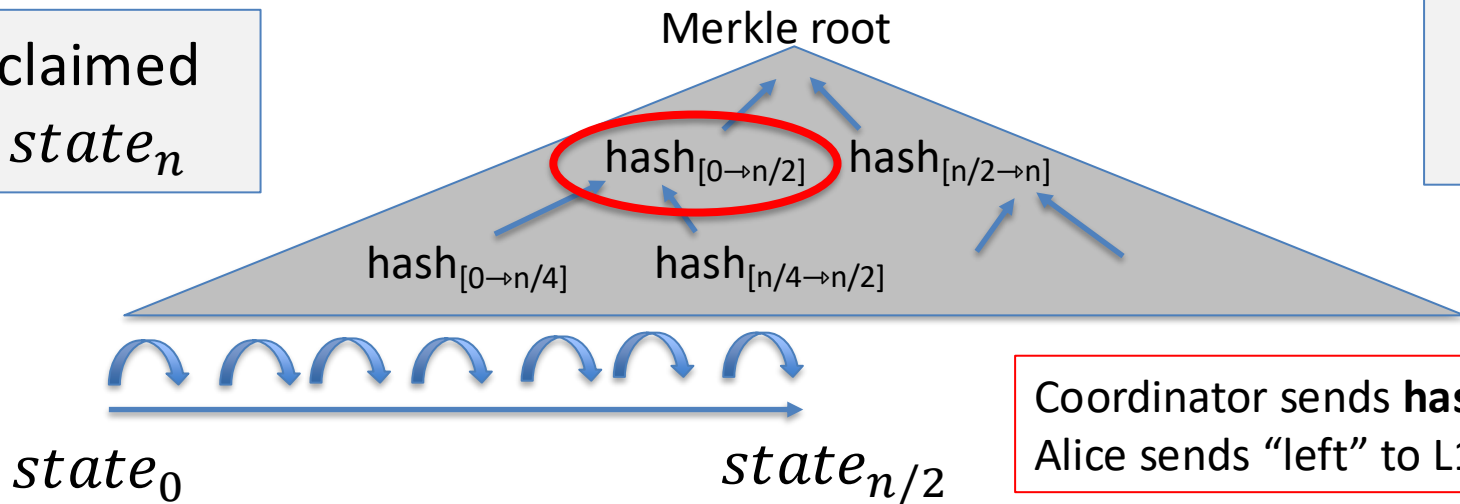
Tx list



fault claim

different
 $state'_n$

Suppose $state_{n/2} \neq state'_{n/2}$



Coordinator sends $hash_{[0 \rightarrow n/2]}$ to L1
Alice sends "left" to L1

Fault Proof game: binary search



coordinator

claimed
 $state_n$

pre-root

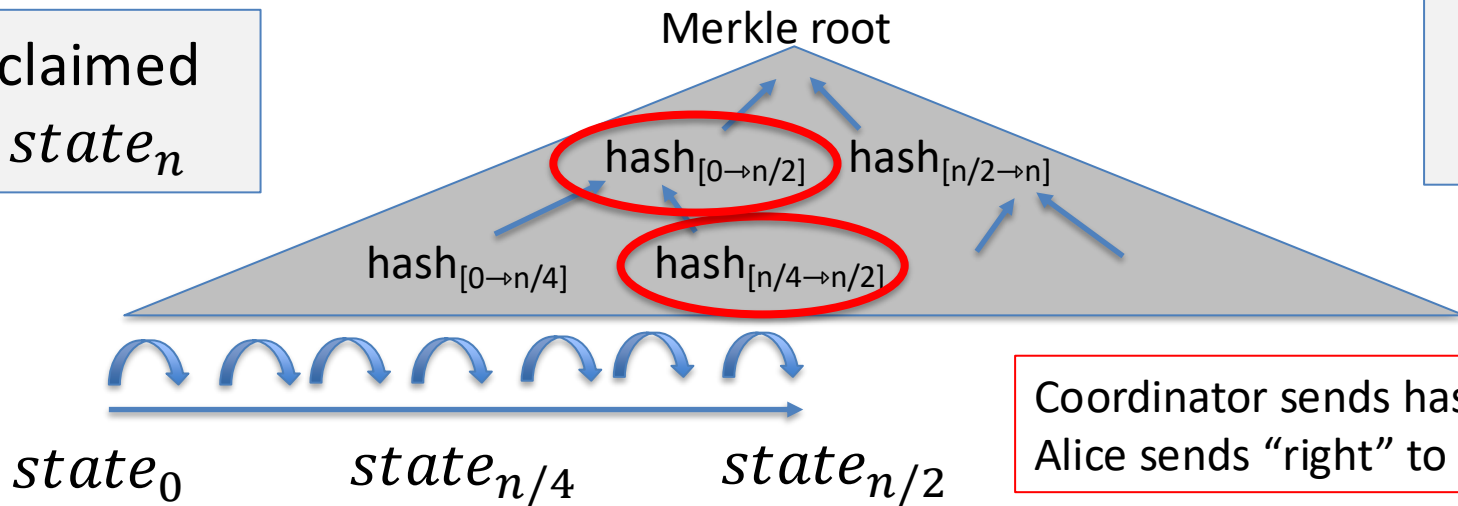
Tx list



fault claim

different
 $state'_n$

Suppose $state_{n/4} = state'_{n/4}$



Coordinator sends hash_[n/4→n/2] to L1
Alice sends "right" to L1

Fault Proof game: binary search



coordinator

claimed
 $state_n$

pre-root

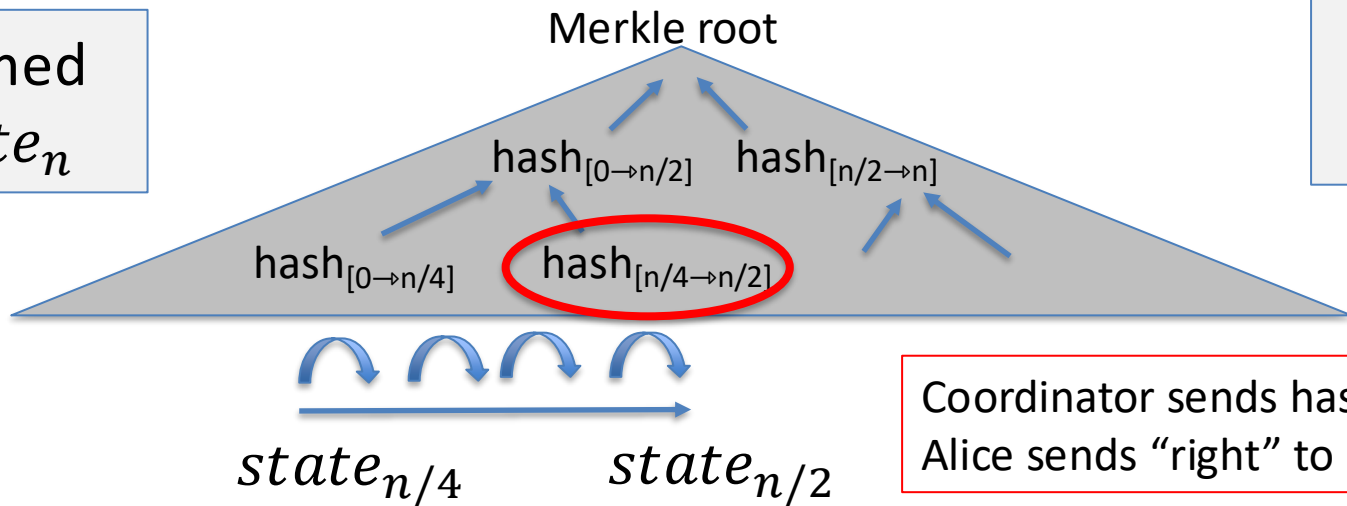
Tx list



fault claim

Suppose $state_{n/4} = state'_{n/4}$

different
 $state'_n$



Coordinator sends $hash_{[n/4 \rightarrow n/2]}$ to L1
Alice sends "right" to L1

Fault Proof game: binary search



coordinator

pre-root

Tx list



After $\log_2 n$ rounds:

- L1 has $state_i$ and $state_{i+1}$ from coordinator
- $state_i = state'_i$ and $state_{i+1} \neq state'_{i+1}$

or game times out because one player defects

⇒ Now L1 can verify fault proof by checking **one** computation step!

Some difficulties with optimistic approach

- (1) Transactions only settle after 7 days (after fault window expires)
 - Alice needs to wait 7 days to withdraw funds from Rollup
(Rollup contract will only send her the funds after 7 days)

[For fungible tokens, a 3rd party can advance the funds to Alice after checking validity of Alice's withdraw Tx. Does not apply to non-fungible tokens.]

- (2) Suppose a successful fault proof 4 days after batch is posted
⇒ all subsequent Tx need to be reprocessed

The end result

Can easily port any smart contract to an optimistic Rollup

- The Rollup EVM can be enhanced with new features (opcodes)

High Tx throughput: in principle, up to 4000 tx/s

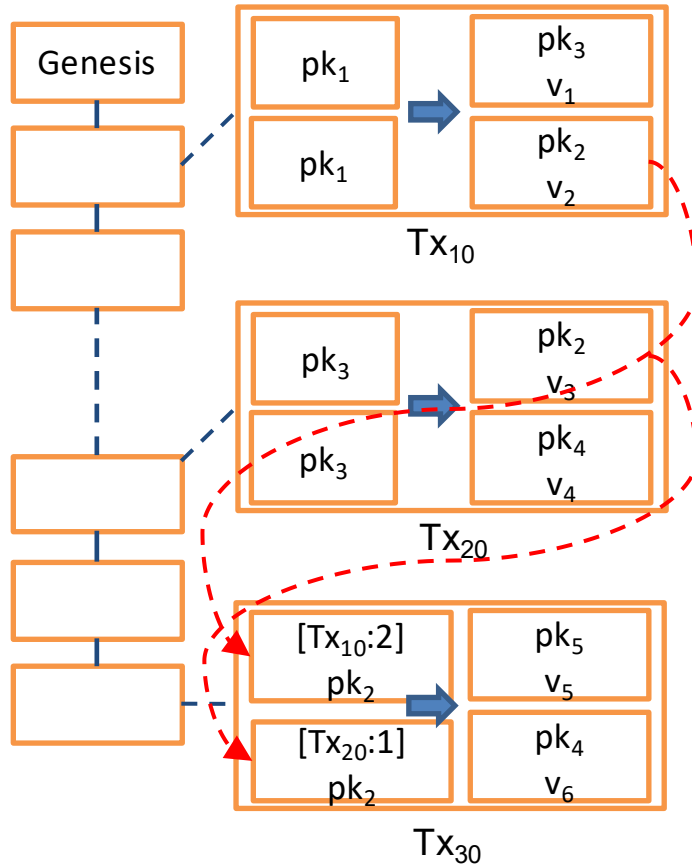
- No need for special hardware at the coordinator

Anyone can act as a coordinator and a verifier

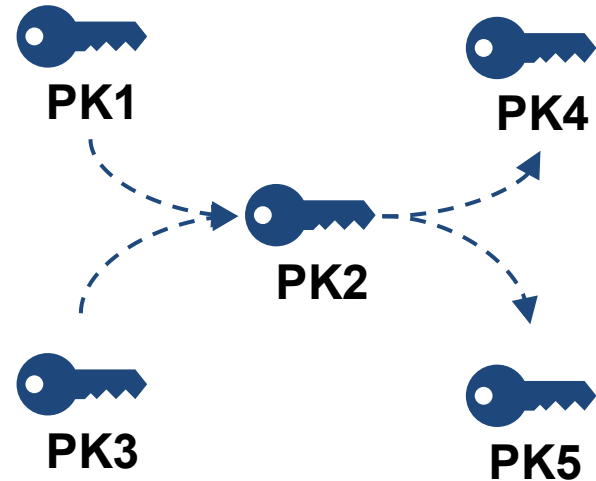
Downside: 7 day finality delay

Data Privacy via ZK Cryptography

UTXO Model

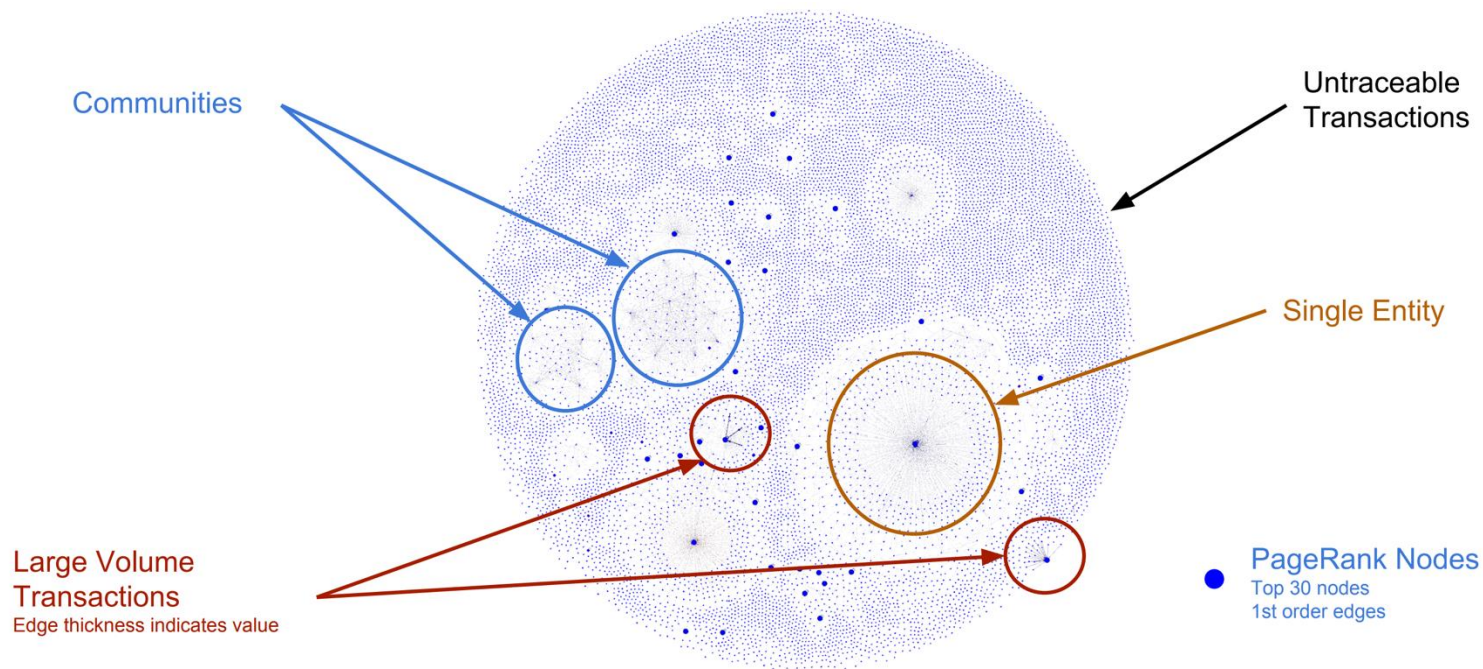


- Bitcoin: Pseudonymous
 - Privacy: public key
 - One can create many public keys



Transaction / public
key graph

Transaction Graph allows Deanononymization



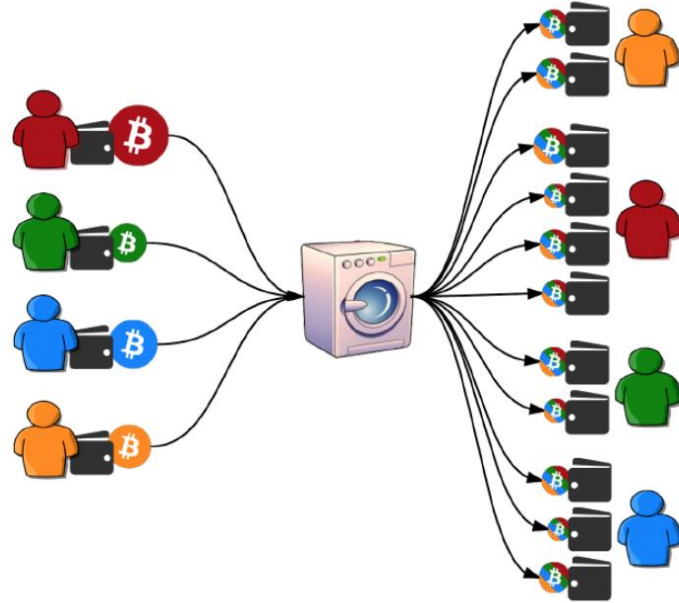
Typical transaction graph for a day

Commercial services built around forensics

- **Chainalysis**
- Used by financial institutions and nation states
- Recover losses from malware attacks
 - Wannacry ransomware, 2017
- Part good (when the “bad guys” are tracked) but
- Part weird (when “routine transactions” are exposed willy-nilly)

Trusted Third-party Mixer

- Laundry service
- Exchanges the coins (the public keys)
- Centralized third-party
 - The mixer can trace / steal the coins
- Example: Coinbase
 - Offers different public keys for each transaction



Zcash (2014)



- Overview: What Is Zcash?
 - **Zcash** is a cryptocurrency that provides **strong privacy guarantees** using **zero-knowledge proofs**, specifically **zk-SNARKs** (Zero-Knowledge Succinct Non-Interactive Argument of Knowledge).
 - It enables **shielded transactions**, where **sender, receiver, and amount** are **entirely encrypted**.
 - Yet, it still allows the network to validate transactions are valid and non-double-spending.
- Use new cryptographic primitives: Zk-SNARK libraries
 - *Zero-knowledge Succinct Non-interactive ARguments of Knowledge*
 - Short and easy to verify

Zcash (2014)

Zcash offers two types of addresses:

1. **Transparent Addresses (t-addrs)** – like Bitcoin addresses.
2. **Shielded Addresses (z-addrs)** – privacy-preserving.

Types of Zcash Transactions

Type	Sender	Receiver	Privacy Level
t → t	Transparent	Transparent	Like Bitcoin (no privacy)
t → z	Transparent	Shielded	Hides receiver and amount
z → t	Shielded	Transparent	Hides sender and amount
z → z	Shielded	Shielded	Full privacy 

Zcash (2014)

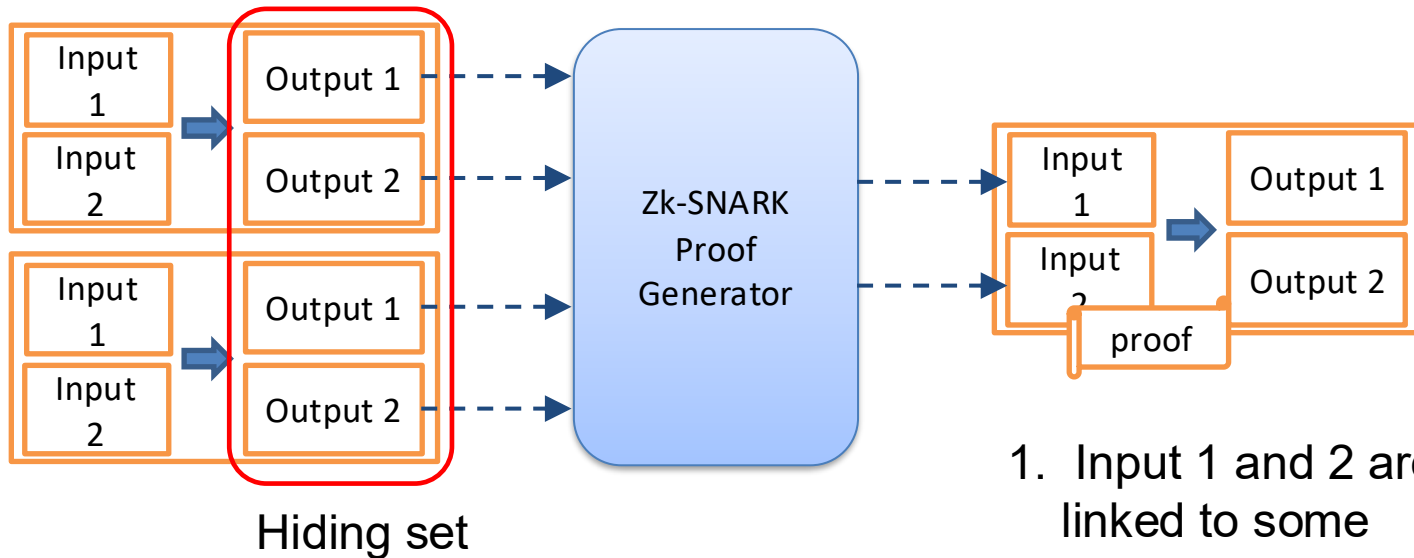
Zero-Knowledge Proofs: zk-SNARKs

Zcash uses **zk-SNARKs** to allow someone to prove that a transaction is **valid** (inputs exist, sums match, not double-spent) **without revealing**:

- Who sent it
- Who received it
- How much was sent

Zcash (2014)

- Special transaction



Zk-SNARK: zero knowledge non-interactive succinct argument of knowledge

1. Input 1 and 2 are linked to some outputs in hiding set
2. Value of input 1,2 is no more than those in the outputs

Language and NP

Definition 1 (Language) A **language** L is a set of statements s.t.

$$L(x) = 1 \text{ if } x \in L$$

Example: x is a number, $L(x)$ is an indicator of whether x is composite

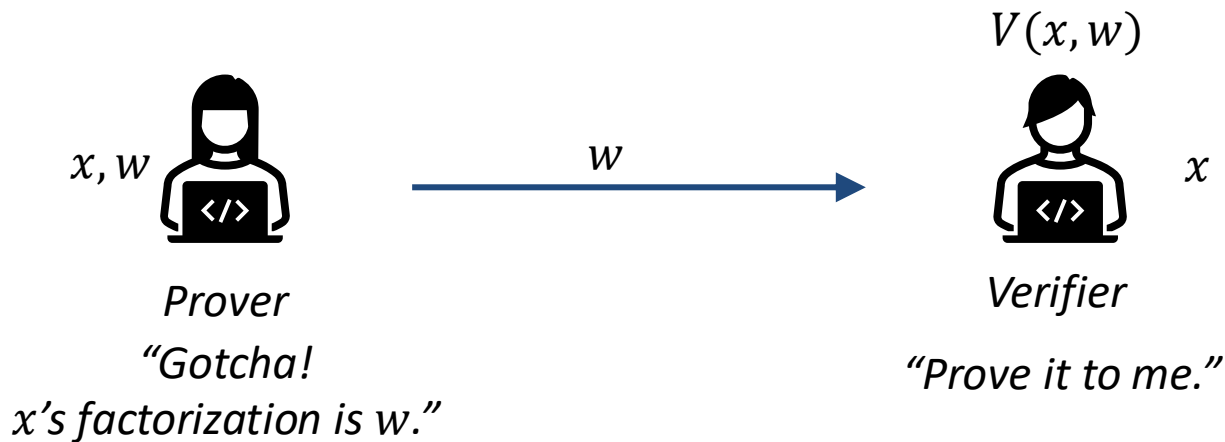
Definition 2 (NP) We define **NP** to be the class of languages L that have a polynomial time **Verifier** V , s.t.

$$L(x) = 1 \iff \exists w, \text{ s.t. } V(x, w) = 1$$

Example: x is a number, $L(x)$ is an indicator of whether x is composite, witness is the prime factorization of x , and verifier can testify the product of w in polynomial time.

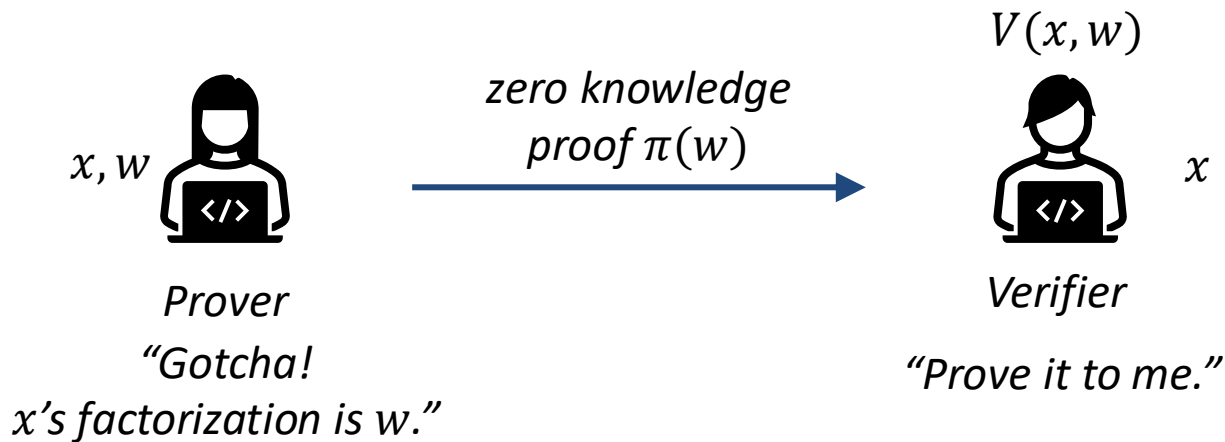
Prover and Zero-knowledge

Given a language L in NP, e.g., verify whether x is a composite.



Prover and Zero-knowledge

Given a language L in NP, e.g., verify whether x is a composite.

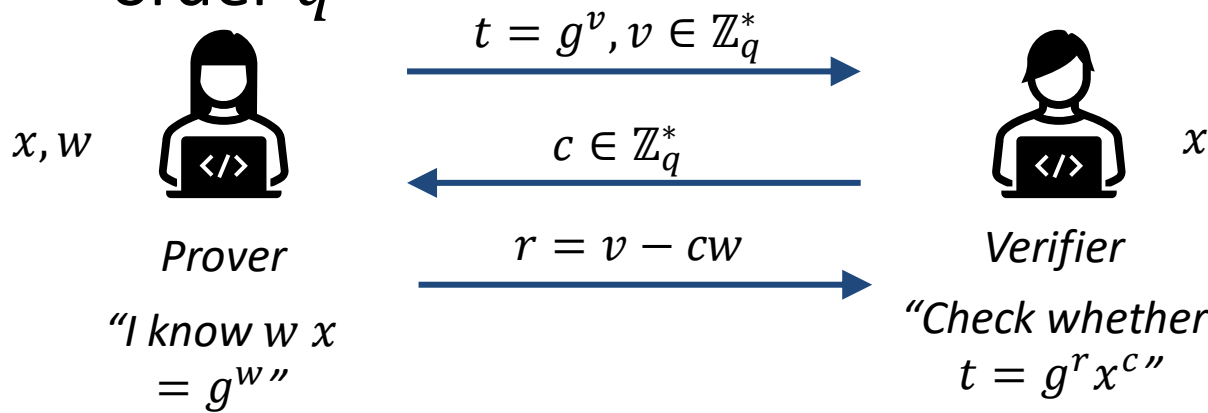


Security Requirements

- Completeness
 - An honest prover with a valid witness can always convince an honest verifier
- Soundness
 - An honest verifier cannot accept a proof if $x \notin L$
- Zero-knowledge
 - The proof does not reveal any information about w

Zero-knowledge Proof

- Schnorr DLOG - Given $x = g^w$, find w .
 - g : a generator of a cyclic group with prime-order q



$$\begin{aligned} g^r x^c &= g^{v-cw} x^c \\ &= g^{v-cw} g^{cw} \\ &= g^v = t \end{aligned}$$

Zero-knowledge Proof

- Schnorr DLOG - Given $x = g^w$, find w .
 - g : a generator of a cyclic group with prime-order q

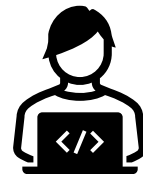
$$\begin{aligned}t &= g^v, v \in \mathbb{Z}_q^* \\ c &= \mathcal{H}(g, x, t) \\ r &= v - cw\end{aligned}$$



Prover

"I know w $x = g^w$ "

(t, r)



Verifier

*"Check whether
 $t = g^r x^c$ "*

x

Zero-knowledge Proof

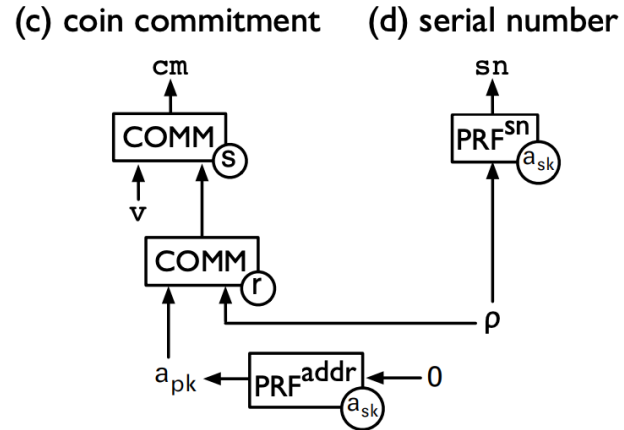
- Time to generate the proof
 - Running time of $V(x, w)$ is T
 - Efficient generator $O(T \log T)$
- Time to verify the proof
 - $O(\log T)$ theoretically
 - Much worse in practice

Zero-knowledge Proof

- Proving process
 - Interactive: earlier example
 - Non-interactive: zk-SNARK / zk-STARK
 - Zk-SNARK: trusted setup
 - Zk-STARK: transparent
- Remarkable Fact:
All languages in NP possess zero-knowledge proofs.

Data Structures

- Address
 - addr_{pk} , addr_{sk}
- Coin c
 - Coin commitment $\text{cmt}(c)$
 - Coin value $v(c)$
 - Coin serial number $\text{sn}(c)$
 - Coin address $\text{addr}_{pk}(c)$
- Pour transaction
 - $(rt, \text{sn}_1^{\text{old}}, \text{sn}_2^{\text{old}}, \text{cmt}_1^{\text{new}}, \text{cmt}_2^{\text{new}}, v_{pub}, \text{info}, \text{proof})$



Problem Statement

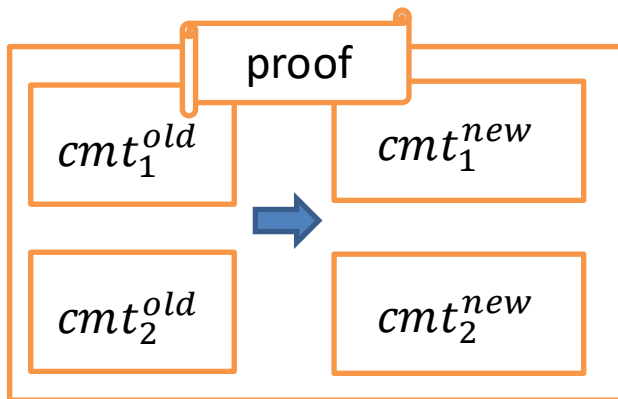
- A pour transaction
$$x = (rt, sn_1^{old}, sn_2^{old}, cmt_1^{new}, cmt_2^{new}, v_{pub}, \text{info}, \text{proof})$$
 - Consume two old coins, c_1^{old}, c_2^{old}
 - Create two new coins, c_1^{new}, c_2^{new}
 - Not reveal the information of coins (e.g., public keys)
- Zk-SNARK proof
 - NP statement *“Given the Merkle-tree root rt , serial number sn_1^{old}, sn_2^{old} , and coin commitments cmt_1^{new}, cmt_2^{new} , I know coins $c_1^{old}, c_2^{old}, c_1^{new}, c_2^{new}$ and address secret key and they satisfy the following conditions...”*

Problem Statement

- A pour transaction x
- $L(x) = 1$: x is a valid transaction
- Witness w
 $= \left(c_1^{old}, c_2^{old}, c_1^{new}, c_2^{new}, \text{addr}_{sk}(c_1^{old}), \text{addr}_{sk}(c_2^{old}) \right)$
- Problem: design the structure of transaction such that no information about the coins is leaked
 - E.g., transaction relationship, public keys, values etc.

First Attempt: commitment

- $x = (cmt_1^{old}, cmt_2^{old}, cmt_1^{new}, cmt_2^{new}, \text{proof})$
- Not reveal values, public keys etc.



First Attempt: commitment

Definition 2 (NP) We define **NP** to be the class of languages L that have a polynomial time **Verifier** V , s.t.

$$L(x) = 1 \iff \exists w, \text{ s.t. } V(x, w) = 1$$

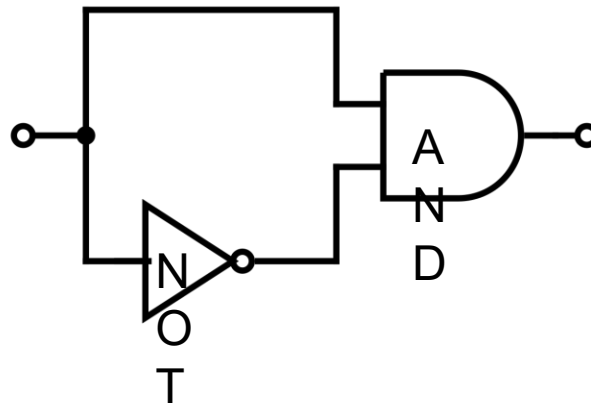
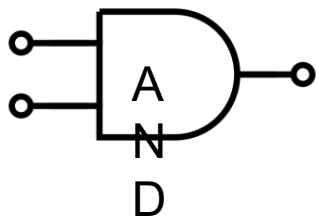
- $x = (cmt_1^{old}, cmt_2^{old}, cmt_1^{new}, cmt_2^{new}, \text{proof})$
- Witness w
 $= (c_1^{old}, c_2^{old}, c_1^{new}, c_2^{new}, \text{addr}_{sk}(c_1^{old}), \text{addr}_{sk}(c_2^{old}))$
- $V(x, w) = 1$ if
 - The commitments of four coins are correct
 - $v(c_1^{old}) + v(c_2^{old}) \geq v(c_1^{new}) + v(c_2^{new})$
 - addr_{sk} of two old coins matches addr_{pk}

First Attempt: commitment

- Zk-SNARK proof
 - *“Given coin commitments cmt_1^{old} , cmt_2^{old} , cmt_1^{new} , cmt_2^{new} , I know coins c_1^{old} , c_2^{old} , c_1^{new} , c_2^{new} and address secret keys and they satisfy the following conditions...”*
 - The sender knows the two old coins and new coins
 - The coins satisfy $v(c_1^{old}) + v(c_2^{old}) \geq v(c_1^{new}) + v(c_2^{new})$
 - The sender has access to $\text{addr}_{\text{sk}}(c_1^{old})$ and $\text{addr}_{\text{sk}}(c_2^{old})$

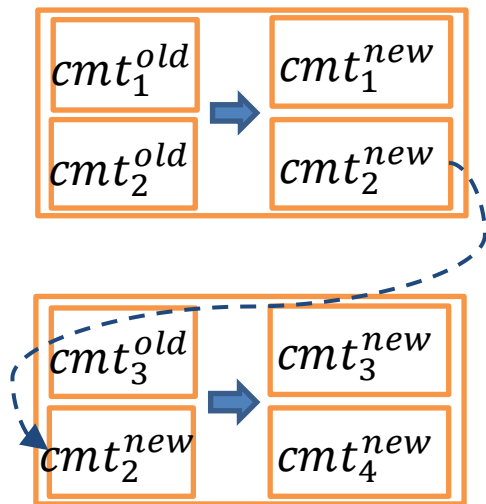
First Attempt: commitment

- Constructions of Zk-SNARK proof
 - Solved by satisfiability circuit
 - A triple of polytime algorithms (KeyGen, Prove, Verify)



First Attempt: commitment

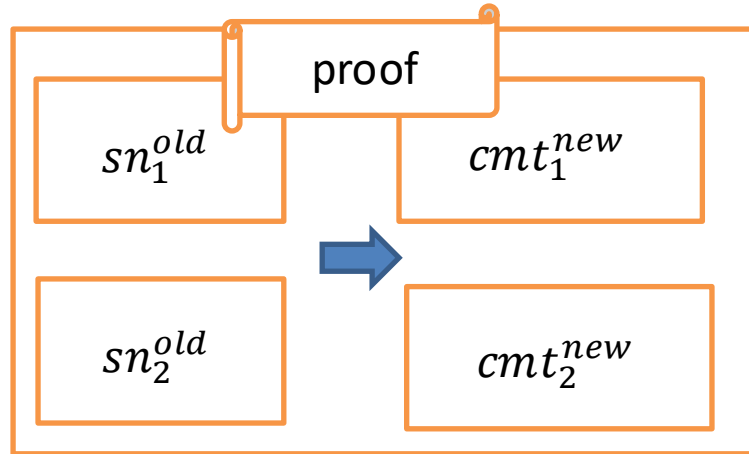
- Anyone who has access to w can create the proof
- Anyone who has access to x can verify the proof



Commitments are still traceable

Second Attempt: two commitments

- Two types of commitments
 - commitment and serial number
- $x = (sn_1^{old}, sn_2^{old}, cmt_1^{new}, cmt_2^{new}, \text{proof}, \text{rt})$

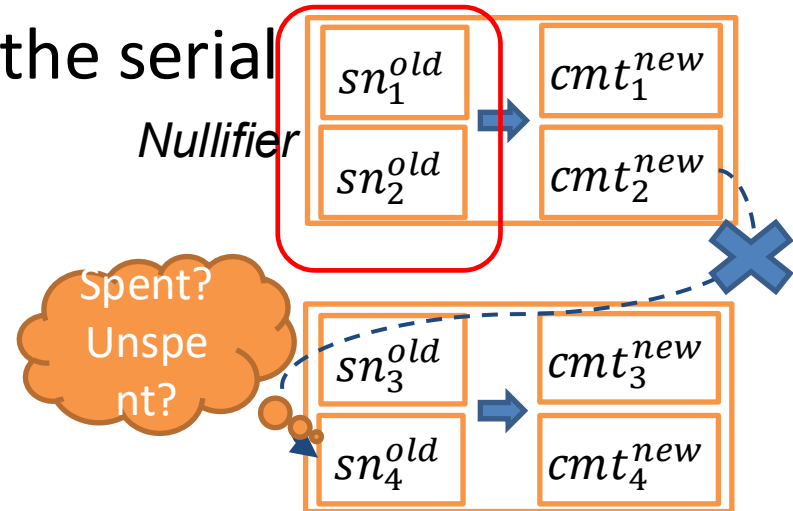


Second Attempt: two commitments

- $V(x, w) = 1$ if
 - The serial numbers of two old coins are correct
 - The commitments of two new coins are correct and verified by Merkle root
 - $v(c_1^{old}) + v(c_2^{old}) \geq v(c_1^{new}) + v(c_2^{new})$
 - addr_{sk} of two old coins matches addr_{pk}

Check Unspent Coins

- How to make sure the old coins are unspent?
 - Keep record of all serial numbers appeared before
 - For new transactions, check if the serial numbers are in the nullifier

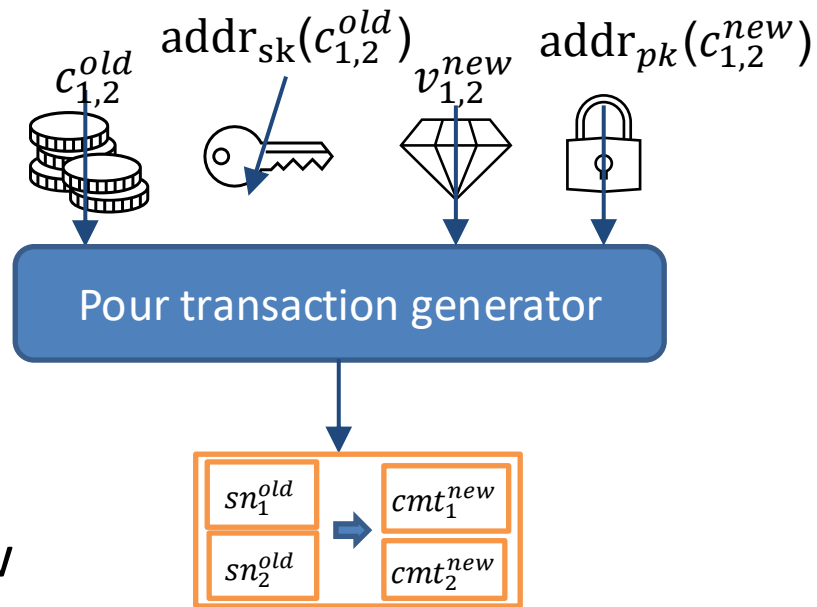


Protocol Summary

- Create pour transaction

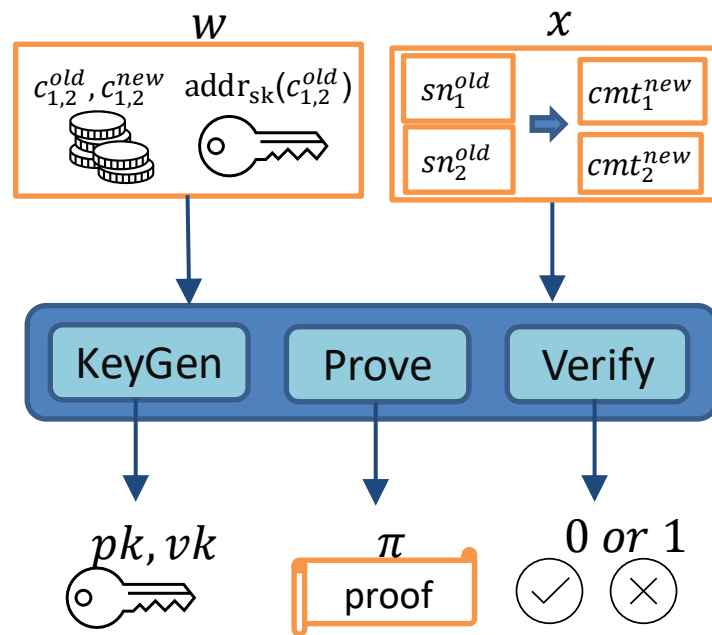
$(rt, sn_1^{old}, sn_2^{old}, cmt_1^{new}, cmt_2^{new}, v_{pub}, info)$

- Two old coins, c_1^{old}, c_2^{old}
- Secret key of two old coins
- New values: v_1^{new}, v_2^{new}
- Public value: v_{pub} , s.t.
$$v(c_1^{old}) + v(c_2^{old}) \geq v(c_1^{new}) + v(c_2^{new}) + v_{pub}$$
- New addresses (public keys of new coins)



Protocol Summary

- Generate the proof
 - Library: zk-SNARK
 - Trusted setup: proving key pk and verifying key vk
 - $\pi = \text{Prove}(pk, w, x)$
 - $\text{Verify}(vk, x, \pi) = 0 \text{ or } 1$



Resources

- ECE/COS 470, Pramod Viswanath, Princeton 2024
- CS251, Dan Boneh, Stanford 2023
- MAS.S62 (Cryptocurrency Engineering and Design), Tadge Dryja, MIT 2018